Lab 1: TypeScript & NestJS Blog API

Complete Solutions & Technical Answers

Part 1: TypeScript Questions & Answers

Q1. What's the difference between let, const, and var?

`var` is function-scoped and can be re-declared and updated. It is hoisted to the top of its scope. `let` is block-scoped, can be updated, but cannot be re-declared in the same scope. `const` is block-scoped and cannot be re-declared or reassigned after initialization.

Q2. What's the difference between interface and type? Give one example for each.

An `interface` is primarily used to define object structures and supports declaration merging. A `type` alias can define primitives, unions, intersections, and tuples, but does not support declaration merging.

```
// Interface Example
interface IUser {
  id: number;
  name: string;
}

// Type Example
type ID = string | number;
type UserType = {
  id: ID;
  name: string;
};
```

Q3. Create an interface for a User object containing: name, email, age, and isAdmin.

```
interface User {
  name: string;
  email: string;
  age: number;
  isAdmin: boolean;
}
```

Q4. Write a function that receives two numbers and returns their sum.

```
function sum(a: number, b: number): number {
  return a + b;
}
```

Q5. Write a function that accepts string | number. If string, return length. If number, return square.

```
function processValue(val: string | number): number {
  if (typeof val === 'string') {
    return val.length;
  }
  return val * val;
}
```

Q6. Create a generic function called firstElement() that returns the first element in any array.

```
function firstElement<T>(arr: T[]): T | undefined {
  return arr[0];
}
```

Q7. Create a User class with name and email and a printInfo() method.

```
class User {
  constructor(public name: string, public email: string) {}

  printInfo(): void {
    console.log(`Name: ${this.name}, Email: ${this.email}`);
  }
}
```

Q8. What's the difference between any and unknown?

`any` disables type checking completely, allowing any property access or method call without safety. `unknown` is a type-safe counterpart; it forces type checking or assertion before performing operations on the variable.

Q9. Create an enum called Role with: Admin, User, and SuperAdmin.

```
enum Role {
  Admin = 'Admin',
  User = 'User',
  SuperAdmin = 'SuperAdmin',
}
```

Q10. Explain the difference between == and === with one example.

`==` performs type coercion before comparison, whereas `===` checks both value and type strictly without coercion.

```
console.log(5 == '5');  // true (string converted to number)
console.log(5 === '5'); // false (different types)
```

Part 2: Blog API Implementation Summary

- **Architecture:** Modular design separating `UsersModule` and `PostsModule`.
- **Database:** MongoDB with Mongoose Schemas and ObjectRef relationships (1 : M).

- **Validation:** DTOs using `class-validator` and `class-transformer` with global `ValidationPipe`.
- **Security:** Password hashing using `bcrypt` and unique email constraints.
- **Features:** Full CRUD operations, cascading post deletion on user removal, populate author details, pagination, search by title, and file uploads.